Tecnologia em Análise e Desenvolvimento de Sistemas - TADS

Estrutura de Dados

Prof. **Luciano Vargas Gonçalves**
E-mail: luciano.goncalves@riogrande.ifrs.edu.br

tads
Análise e Desenvolvimento de Sistemas
INSTITUTO FEDERAL
Rio Grande do Sul
Campus Rio Grande

Sumário

Estrutura de Dados

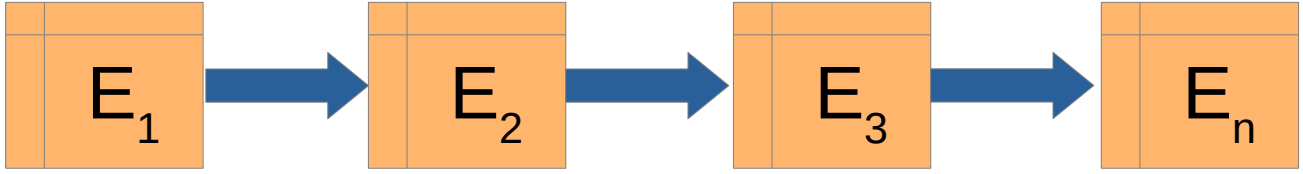
- Listas Duplamente Encadeadas - LDE

Sumário

- **Estrutura de Dados**
  - **Dinâmicas:**
    - O tamanho se altera com a necessidade;
    - Cresce ou Decresce
      - Listas
        - Simplesmente Encadeadas
        - **Duplamente Encadeadas**
      - Pilhas
      - Filas

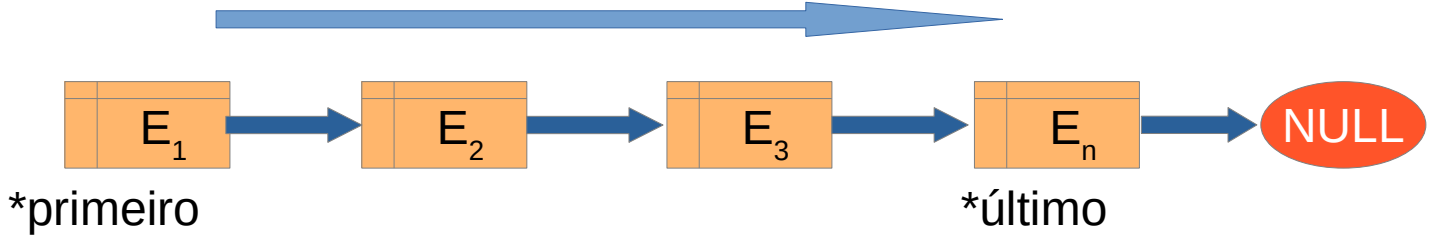
Listas

- **Listas**
  - É uma coleção de elementos (Nó) do mesmo tipo, dispostos linearmente, que podem ou não seguir determinada organização, por exemplo:
    - [E1,E2,E3,E4,E5,....,En]
  - Onde n seja n >= 0;

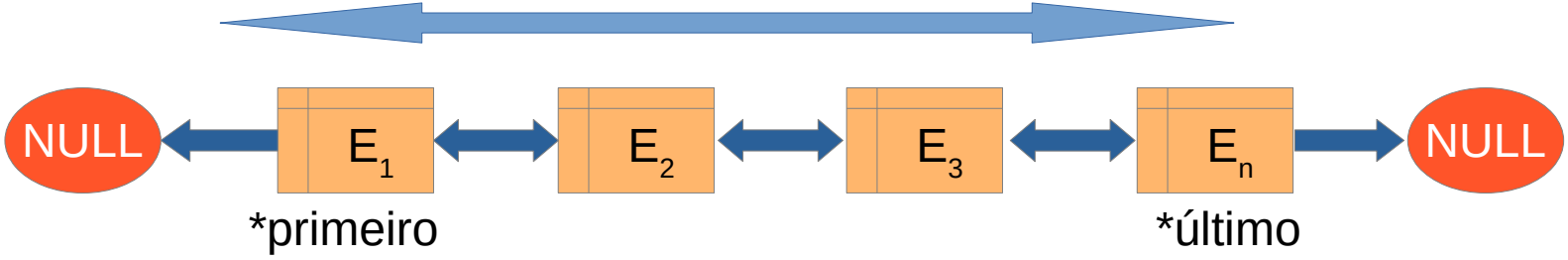


Listas

- **Listas Encadeadas:**
  - Simples (Deslocamento (acesso) em um sentido)

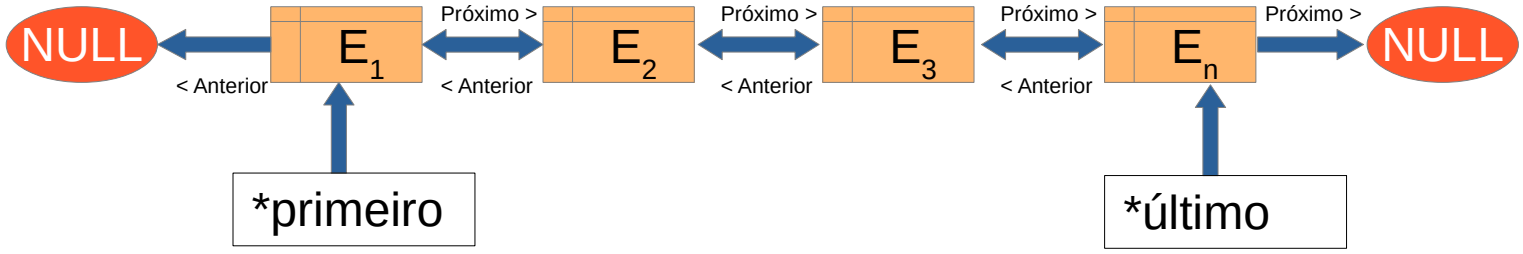


  - Duplamente (Deslocamento em ambos sentidos)



Listas Duplamente Encadeada

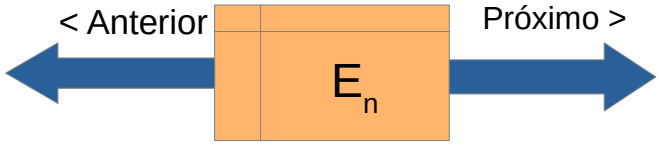
- Cada elemento (nó) armazenar uma referência para o **anterior e o próximo** elemento, sendo o elo de ligação dos elementos;
- Restrições:
  - Acesso pelo Extremo (Primeiro / Último);
  - Inserção e remoção em qualquer posição da Lista



Lista Duplamente Encadeada

Elemento de Lista - E_n

- O elemento (E_n) é uma estrutura de dados que irá armazenar as informações:
  - Informação do Elemento (dados);
  - Apontadores para os elementos ***Anterior e Próximo*** (Ponteiros *)



Elemento de armazenamento

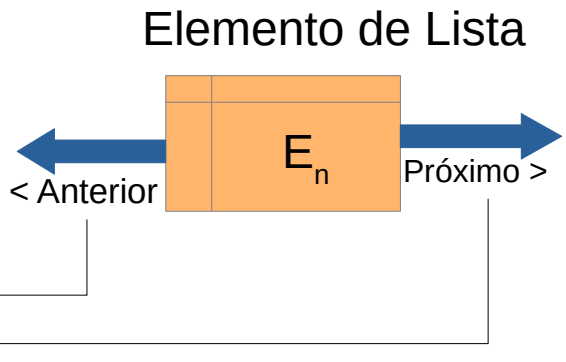
Elemento de Lista - E_n

- Elemento de Lista:
  - Contará com os dados;
  - Ponteiros para Anterior e Próximo;
  - Exemplo:
    - Dados de uma tarefa;
      - Descrição;
      - Prioridade;
      - Id
      - Concluída.

```
typedef struct tarefa
{
    char descricao[30];
    int id, prioridade, concluida;
    struct tarefa *anterior;
    struct tarefa *proximo;
} Tarefa;
```

Elemento de Lista - E_n

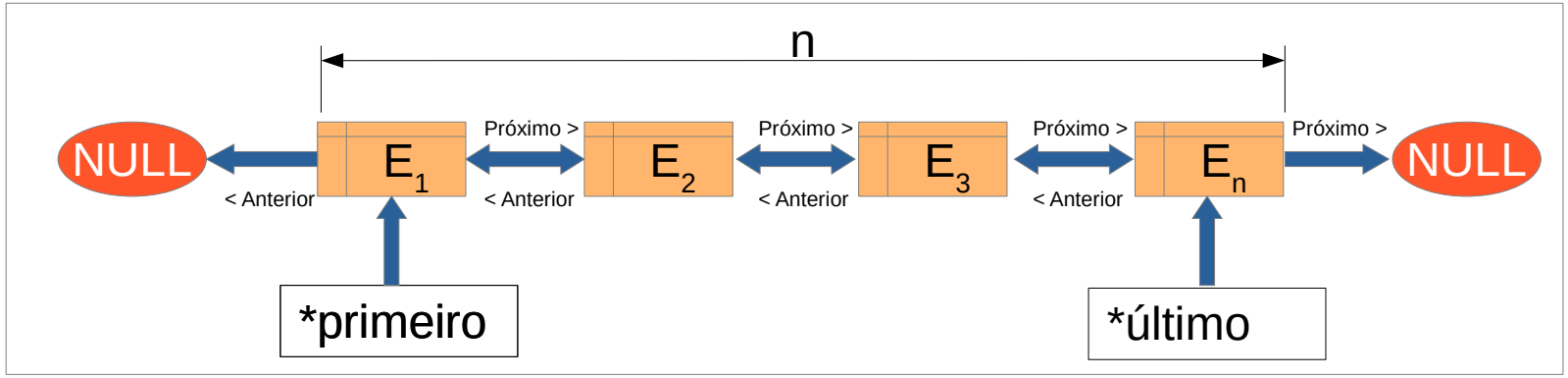
```
typedef struct tarefa
{
    char descricao[30];
    int id, prioridade, concluida;
    struct tarefa *anterior;
    struct tarefa *proximo;
} Tarefa;
```



Estrutura de Lista Encadeada

- Estrutura da Lista:
  - Precisamos armazenar:
    - Apontador para “PRIMEIRO”;
    - Apontador para ÚLTIMO elemento;
    - Contador de Elementos (n);

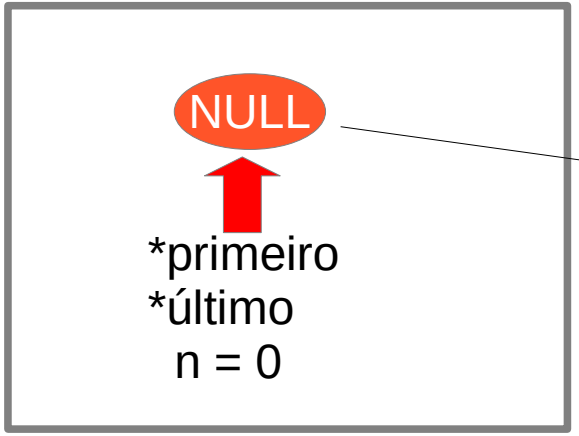
```
typedef struct LDE
{
    Tarefa *primeiro;
    Tarefa *ultimo; // facultativo
    char nome[30];
    int num;
} LDE;
```



Lista Duplamente Encadeada

Cria uma Lista Vazia

- **Criar uma Estrutura de Lista Vazia:**
  - Precisamos inicializar o apontador “PRIMEIRO”;
  - Inicializar o contador de Elementos (n);



*NOVA - Lista LDE

Definir a Lista

```
LDE *criaListaLDE(char nome[])
{
    // Aloca memória e inicializa uma nova lista LDE
    LDE *nova = (LDE *)malloc(sizeof(LDE));
    strcpy(nova->nome, nome);
    nova->primeiro = NULL;
    nova->ultimo = NULL;
    nova->num = 0;
    return nova;
}
```

Criar e Inicializar a Lista Duplamente Encadeada

Elemento – Alocação e Inicialização

- Função para criar uma nova tarefa

```
Tarefa * criaTarefa(int id){
    Tarefa *nova = (Tarefa*)malloc(sizeof(Tarefa));
    nova->id = id;
    fflush(stdin);
    printf("Informe sua Prioridade:");
    scanf("%d",&nova->prioridade);
    printf("Informe a Descricao:");
    scanf("%s",nova->descricao);
    printf("Tarefa Criada com Sucesso\n ");
    return nova;
}
```

Funções para controle da Lista

- Funções de Inserção

  Início, Fim, Posição qualquer (n)
- Funções de Remoção
  - *Início, Fim, Posição qualquer (n)*
- Funções para Mostrar a lista
  - MostraListaEsquerdaDireita( ) //navegando pelo ponteiro *proximo
  - MostraListaDireitaEsquerda( ) //navegando pelo ponteiro *anterior
- Funções para Liberar memória
  - ApagaElemento( ) e ApagaLista( )

Funções para controle da Lista

- Funções de Inserção
  - Recebe um ponteiro de Lista e um Ponteiro de Elemento
    - InsereNoInicio(Lista *ls , Tarefa *tf)
      - Insere uma nova tarefa na primeira posição da Lista (E_0)
    - InsereNoFim(Lista *ls , Tarefa *tf)
      - Insere uma nova tarefa na última posição da Lista (E_n)
    - InsereNaPosicao(Lista *ls , Tarefa *tf , int p)
      - Insere uma nova tarefa em uma posição determinada, o elemento ocupa a posição (p)

Funções para controle da Lista

- Funções de Remoção
  - Recebe um ponteiro de Lista e retorna um elemento da lista
  - Tarefa * RemoveNoInicio(Lista *ls)
    - Remove a tarefa da primeira posição da Lista (E_0)
  - Tarefa * RemoveNoFim(Lista *ls)
    - Remove a tarefa na última posição da Lista (E_n)
  - Tarefa * RemoveNaPosicao(Lista *ls)
    - Remove a tarefa que está na posição (p)

Funções para controle da Lista

- Funções para Mostrar a lista
  - MostraListaEsquerdaDireita(Lista ls)
    - Navegando do ponteiro ***primeiro*** até o último elemento através do ponteiro ***próximo***;
  - MostraListaDireitaEsquerda(Lista ls)
    - Navegando do ponteiro ***Último*** até o primeiro elemento através do ponteiro ***anterior***;

Funções para controle da Lista

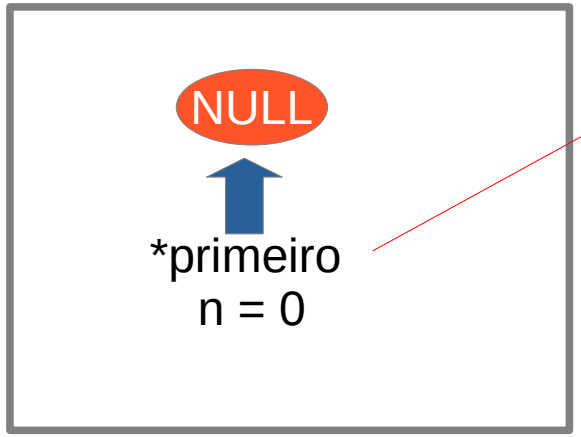
- Funções para Liberar memória
  - ApagaElemento(Tarefa *tf)
    - Libera a memória ocupada por um elemento de lista (free elemento)
  - ApagaLista(Lista *ls)
    - Libera a memória de todos elementos de lista (free e cada elemento)

Insere Elemento Início

- **Método para inserir um elemento no *Início* da Lista**
  - Estado da Lista - Possibilidades :
    - Lista vazia
    - Lista com elementos
  - Protótipo da Função
    - Recebe um ponteiro para lista e um ponteiro de elemento;
    - Função sem retorno;
      - void insereNoInicio(LDE *l,Tarefa *tf);

Insere Elemento Início

- **Método para inserir um novo elemento no *Início* da Lista**
  - Primeiro Passo: Lista Vazia
    - *Ponteiro primeiro* aponta para NULL;



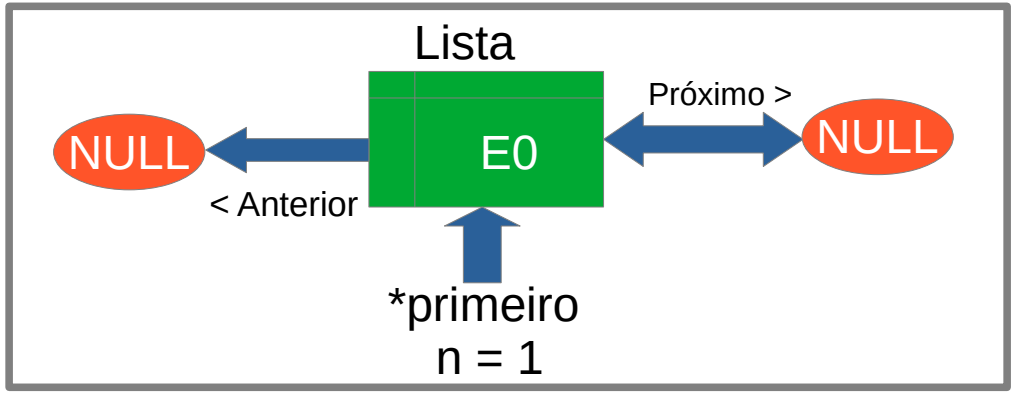
Lista Vazia

```
void insereInicio(LDE *lt, Tarefa *tf)
{
    // Insere um elemento no inicio da lista LDE
    tf->anterior = NULL;
    if (lt->primeiro == NULL)
    {
        tf->proximo = NULL;
        lt->ultimo = tf;
    }
    else
    {
        tf->proximo = lt->primeiro;
        lt->primeiro->anterior = tf;
    }
    lt->primeiro = tf;
    lt->num++;
}
```

Insere no Início da Lista Vazia

Insere Elemento Início

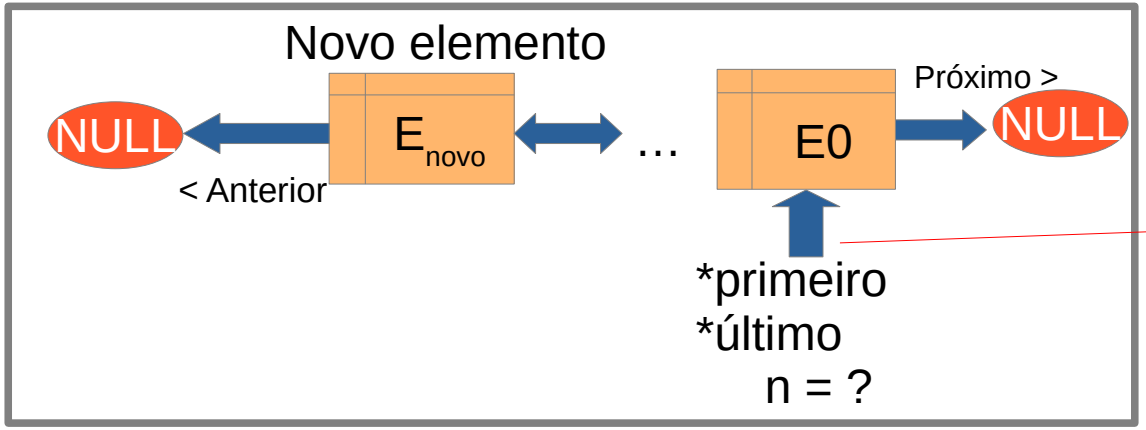
- **Método para inserir um elemento no *Início* na Lista**
  - Após a Inserção a lista ficará com a seguinte configuração



Lista com 1 elemento após inserção

Insere Elemento Início

- **Método para inserir um elemento no *Início* da Lista**
  - Segundo Passo: lista com 1 ou mais elementos
  - Ponteiro para primeiro aponta para um elemento;

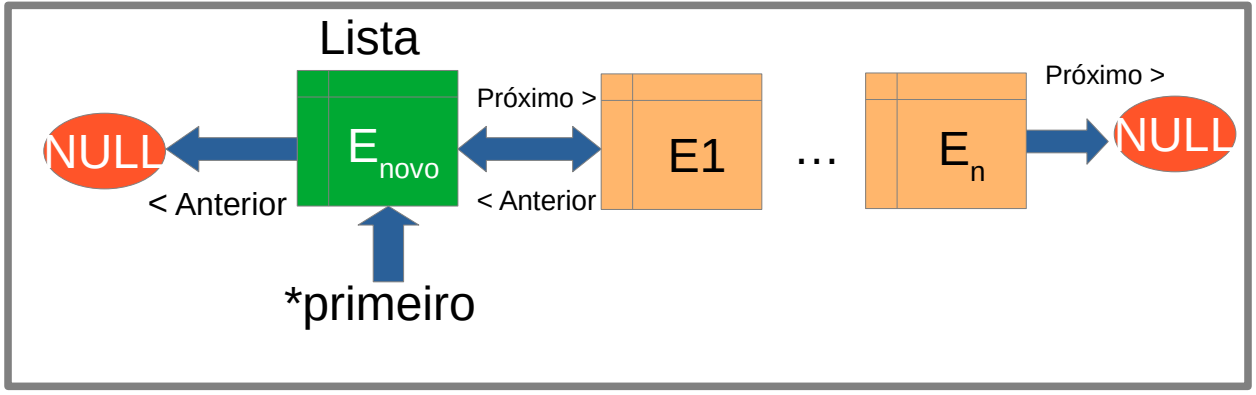


Lista com 1 ou mais elementos

```
void insereInicio(LDE *lt, Tarefa *tf)
{
    // Insere um elemento no inicio da lista
    tf->anterior = NULL;
    if (lt->primeiro == NULL)
    {
        tf->proximo = NULL;
        lt->ultimo = tf;
    }
    else
    {
        tf->proximo = lt->primeiro;
        lt->primeiro->anterior = tf;
    }
    lt->primeiro = tf;
    lt->num++;
}
```

Insere Elemento Início

- **Método para inserir um elemento no *Início* da Lista**
  - Após a Inserção a lista ficará com a seguinte configuração



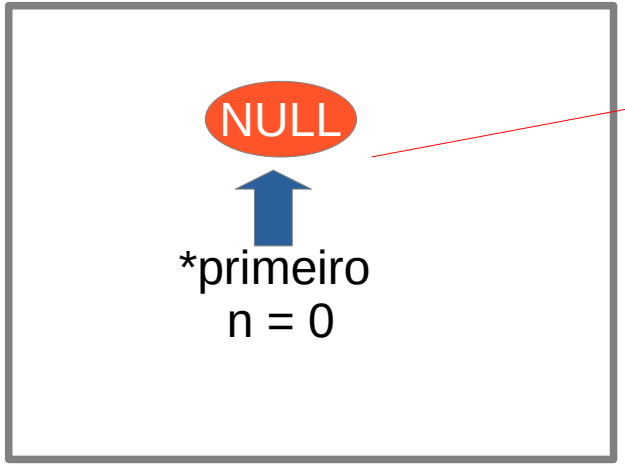
Lista com 1 ou mais elementos

Insere Elemento Fim

- **Método para inserir um elemento no *FIM* da Lista**
  - Estado da Lista - Possibilidades :
    - Lista vazia
    - Lista com elementos
  - Protótipo da Função
    - Recebe um ponteiro para lista e um ponteiro de elemento;
    - Função sem retorno;
      - void insereNoFim(LDE *l,Tarefa *tf);

Insere Elemento Fim

- **Método para inserir um elemento no *Fim* da Lista**
  - Primeiro Passo: Lista Vazia
    - Processo igual ao insereNoFim**;**

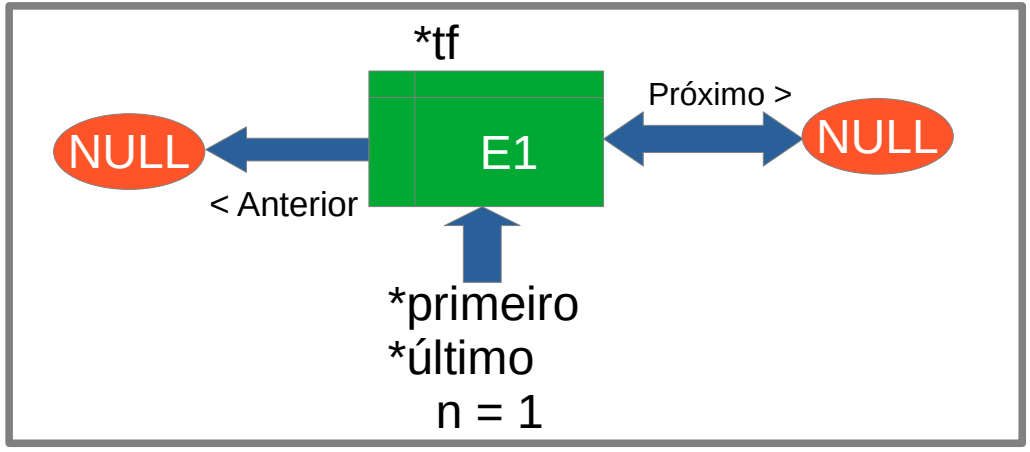


Lista estado atual

```c
void insereFim(LDE *lt, Tarefa *tf)
{
    // Insere um elemento no Fim da lista LDE
    tf->proximo = NULL;
    if (lt->primeiro == NULL)
        insereInicio(lt, tf);
    else
    {
        tf->anterior = lt->ultimo;
        lt->ultimo->proximo = tf;
        lt->ultimo = tf;
        lt->num++;
    }
}
```

Insere Elemento Fim

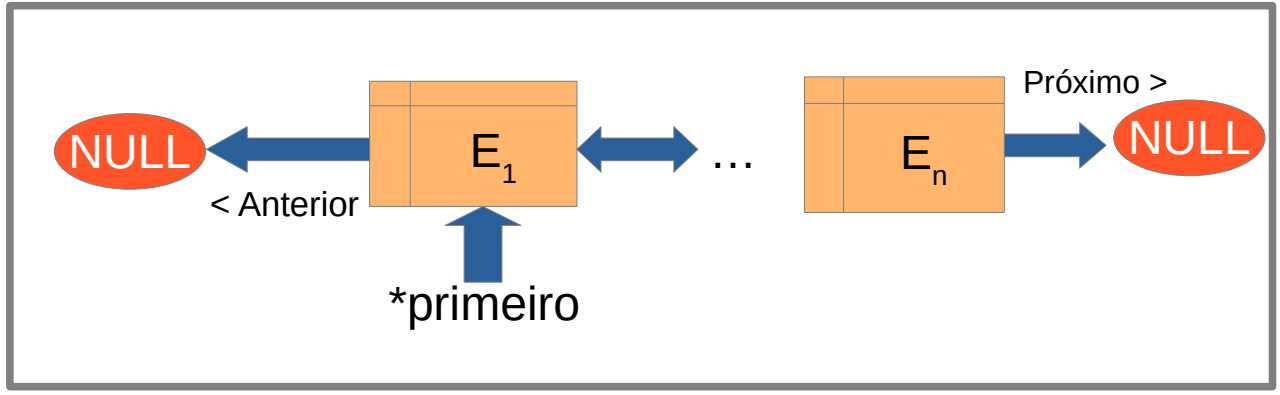
- **Método para inserir um elemento no *Fim* na Lista**
  - Após a Inserção a lista ficará com a seguinte configuração



Lista com 1 (uma) tarefa

Insere Elemento Fim

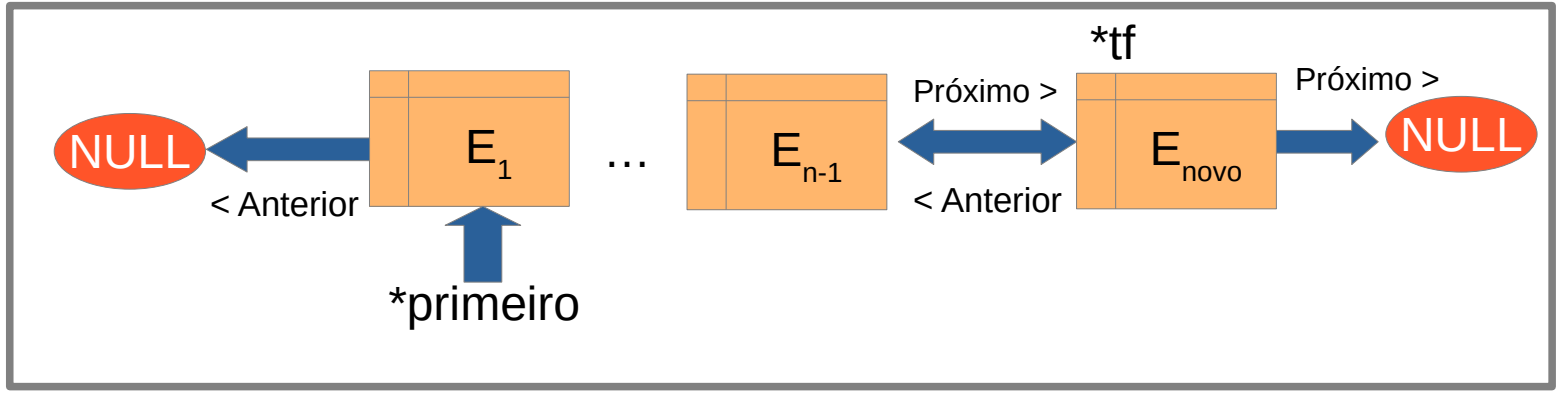
- **Método para inserir um elemento no *Fim* da Lista**
  - Segundo Passo: lista com 1 ou mais elementos
    - Ponteiro para primeiro aponta para um elemento;



Lista com 1 ou mais elementos

Insere Elemento Fim

- **Método para inserir um elemento no *Fim* da Lista**
  - Processo de navegação para encontrar último elemento da lista



Lista com 1 ou mais elementos

Inserção Otimizada

- Avaliação do ChatGpt – Como Otimizada
  - Complexidade θ(1)

✔ **Inserção no fim otimizada**

```c
lt->ultimo->proximo = tf;
```

👉 Usa o ponteiro `ultimo` → **O(1)** (excelente decisão)

Insere Elemento Fim

- **Método para inserir um elemento no *Fim* da Lista**
  - Busca pelo último elemento da lista

```c
void insereNoFim(LDE *l, Tarefa *tf){
    printf("\nInsere no FIM\n");
    Tarefa *aux = l->primeiro;
    if(l->primeiro == NULL)
        insereNoInicio(l,tf);
    else{
        while(aux->proximo != NULL){
            aux = aux->proximo;
        }
        aux->proximo = tf;
        tf->anterior = aux;
        tf->proximo = NULL;
        l->n++;
    }
}
```

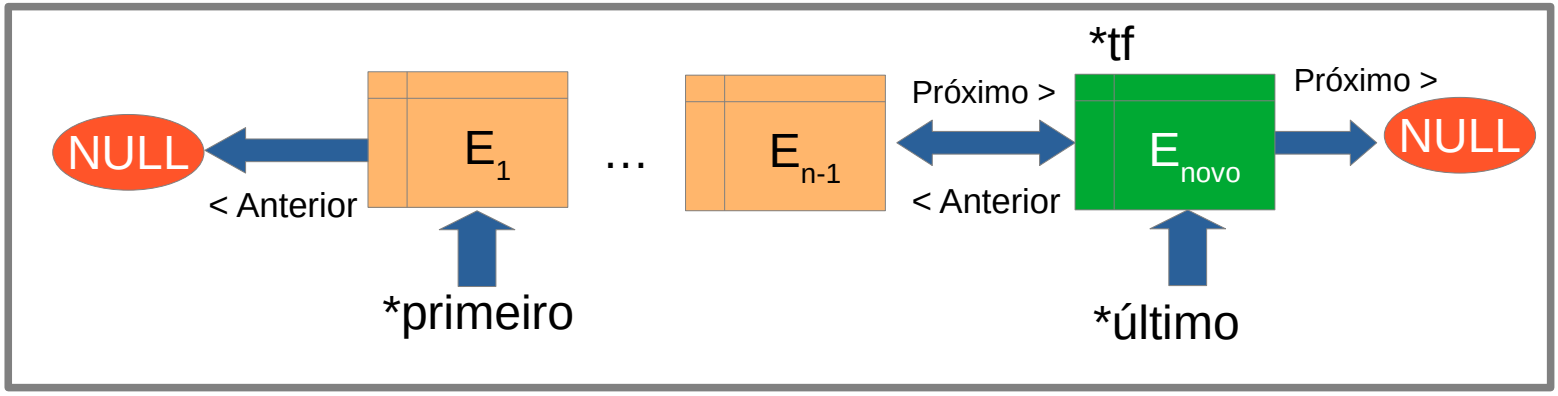
Lista sem *último - θ(n)

```c
void insereFim(LDE *lt, Tarefa *tf)
{
    // Insere um elemento no Fim da lista LDE
    tf->proximo = NULL;
    if (lt->primeiro == NULL)
        insereInicio(lt, tf);
    else
    {
        tf->anterior = lt->ultimo;
        lt->ultimo->proximo = tf;
        lt->ultimo = tf;
        lt->num++;
    }
}
```

Lista com *último - θ(1)

Insere Elemento Fim

- **Método para inserir um elemento no *Fim* da Lista**
  - Processo de navegação para encontrar último elemento da lista



Lista com 1 ou mais elementos, sendo o último elemento inserido no Fim

Lista sem *último

Mostra Lista Esquerda para Direita

- Mostrar a lista a partir do ponteiro primeiro;
  - Navegar (Mostrar) pelo ponteiro *proximo

```
void mostraListaED(LDE lt){
    //Mostrar Lista da Esquerda para Direita -
      //através do ponteiro Proximo
    Tarefa *aux = lt.primeiro;
    if(aux == NULL)
        printf("\n LISTA VAZIA!");
    else{
        printf("\n Início da Lista!");
        while(aux != NULL){
            mostraTarefa(*aux);
            aux = aux->proximo;
        }
        printf("\n Fim da Lista ED!");
    }
}
```

Mostra Lista Direita para Esquerda

- Mostrar a lista a partir do ponteiro último;
  - Navegar (mostrar) pelo ponteiro *anterior

```
void mostraListaDE(LDE lt){
    //Mostra lista Ultimo para o Primeiro -
      //através do ponteiro anterior
    Tarefa *aux = lt.ultimo;
    if(aux == NULL)
        printf("\n LISTA VAZIA!");
    else{
        printf("\n Início da Lista!");
        while(aux != NULL){
            mostraTarefa(*aux);
            aux = aux->anterior;
        }
        printf("\n Fim da Lista DE!");
    }
}
```

Listas Duplamente Encadeada

- **A Lista duplamente encadeada terá as funções:**
  - Insere um elemento no início da lista;
  - Insere um elemento em uma posição definida;
  - Conta Elementos;
  - Mostra Lista;
  - Remove e retorna o primeiro elemento da lista;
  - Remove e retorna o último elemento da lista;
  - Remove e retorna o elemento na posição definida;
  - Mostra Lista da Esquerda para Direita;
  - Mostra Lista da Direita para Esquerda;

Listas Duplamente Encadeada

Protótipo das funções

```
LDE* criaListaLDE();
Tarefa* criaTarefa(int id);

void insereNoInicio(LDE *l, Tarefa *tf);
void insereNoFim(LDE *l, Tarefa *tf);
int insereNaPosicao(LDE *l, Tarefa *tf, int p);

Tarefa * removeNoInicio(LDE *l);
Tarefa * removeNoFim(LDE *l);
Tarefa * removeNaPosicao(LDE *l, int p);

void mostraDados(Tarefa d);

void mostraListaED(LDE *l);//mostra da Esquerda para Direita
void mostraListaDE(LDE *l);//mostra da Direita para Esquerda

void mostraTarefaPosicao(LDE *l, int posicao);

void apagaLDE(LDE *l);

void menu(LDE *l);
```

Programa principal (main)

- Um ponteiro de Lista *mLista
- Função criaListaLDE para alocar e inicializar a lista;
- Função menu de lista para testar

```
int main()
{
    LDE *mLista;
    mLista = criaListaLDE("MINHAS TAREFAS");
    menu(mLista);
}
```

Função Menu

Chamada das Funções

```
void menu(LDE *l){
    int op, posicao, id=1;
    Tarefa *tf;
    char ch;
    do{
        //system("clear");
        printf("\n\nInforme uma Opção:");
        printf("\n -- 1 - para Inserir Tarefa no Início:");
        printf("\n -- 2 - para Inserir Tarefa no Fim:");
        printf("\n -- 3 - para Inserir Tarefa na Posição:");
        printf("\n -- 4 - para Remove Tarefa no Início:");
        printf("\n -- 5 - para Remove Tarefa no Fim:");
        printf("\n -- 6 - para Remove Tarefa na Posição:");
        printf("\n -- 7 - para Mostra um Tarefa da Posição:");
        printf("\n -- 8 - para Mostrar a Lista ED:");
        printf("\n -- 9 - para Mostrar a Lista DE:");
        printf("\n -- 10 - Apagar Lista:");
        printf("\n -- 0 - para Sair do Programa:\n");
        printf("\nInforme sua Opçao:");
        scanf("%d",&op);
        fflush(stdin);
```

Execução do programa

- Opções para Lista de tarefas

```
Informe uma Opção:
 -- 1 - para Inserir Elemento no Início:
 -- 2 - para Inserir Elemento no Fim:
 -- 3 - para Inserir Elemento na Posição:
 -- 4 - para Remove Elemento no Início:
 -- 5 - para Remove Elemento no Fim:
 -- 6 - para Remove Elemento na Posição:
 -- 7 - para Mostra um Elemento da Posição:
 -- 8 - para Mostrar a Lista ED:
 -- 9 - para Mostrar a Lista DE:
 -- 10 - Apagar Lista:
 -- 0 - para Sair do Programa:

Informe sua Opçao:
```

Execução do programa

- Algumas tarefas do Dia

```
---------- Lista de MINHAS TAREFAS -----------------

Início da Lista!
Tafera N° 1
        Descricao: Acordar
        Prioridade: 1
        Tarefa Não concluída.
Tafera N° 2
        Descricao: Escovar_Dentes
        Prioridade: 2
        Tarefa Não concluída.
Tafera N° 3
        Descricao: Tomar_Café
        Prioridade: 3
        Tarefa Não concluída.
Tafera N° 4
        Descricao: Ir_trabalho
        Prioridade: 3
        Tarefa Não concluída.
Fim da Lista ED!
```

Exercícios

1) Implementar as funções que estão em Branco;
    1) InserePosição, Remove Início, RemoveFim, RemovePosição
2) Testar a criação de uma lista com várias tarefas agendadas;
    - Realizar várias inserções e remoções.
3) Mostrar as atividades cadastradas.
    - Encerrar o programa;

- Dúvidas !!!